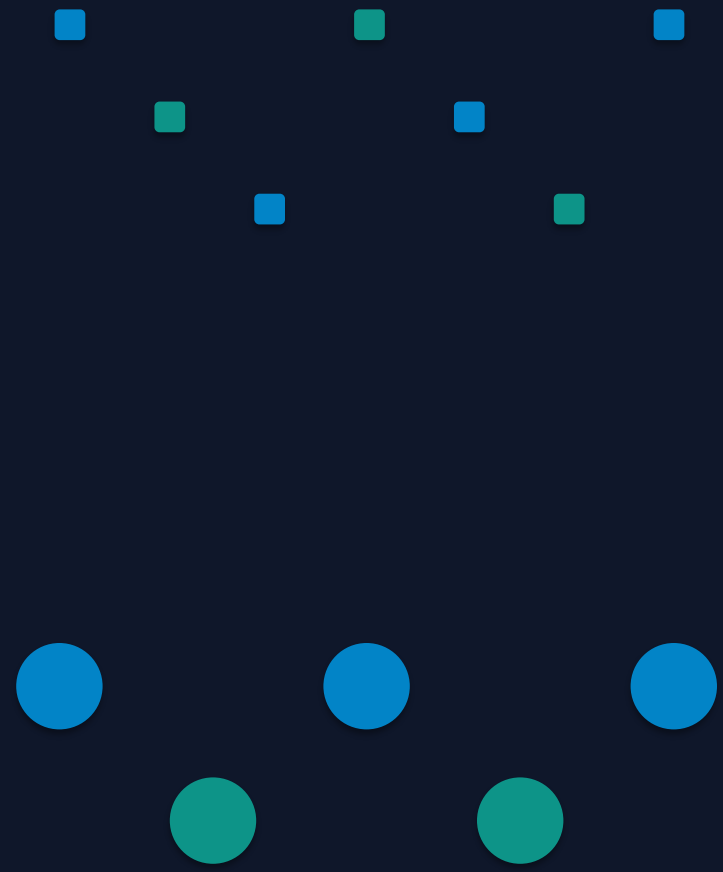
PROJECT PRESENTATION • 2026–27

SmartCare

Smart Hospital Management System

A unified digital platform for hospital administration and clinical workflows.

Team Nobita | Department of AI & Data Science
Arya College of Engineering & I.T. | RTU Kota



JAVA • JSP / SERVLETS • MYSQL

Guide: Er. Ram Babu Buri • HOD: Dr. Ashok Kumar Kajla

Project at a glance

SmartCare brings connected hospital operations into one role-based application.

Purpose

Organize patient and hospital workflows in a single system.

Users

Administrators, doctors, patients, nurses, and receptionists.

Core idea

Use structured records and controlled access to support coordinated care.

Project technology

- Java 17
- Jakarta Servlet/JSP
- JDBC
- MySQL
- Maven WAR
- BCrypt

Design goals

- Clear role-based workspaces
- Consistent, relational records
- Secure sign-in and session handling
- Easier access to operational information

Project details summarized from the provided source archive and documentation.

Problem statement

Disconnected records and manual coordination can slow down everyday hospital work.

Fragmented information

Patient, appointment, and billing details can be stored separately.

Coordination overhead

Staff need to coordinate schedules, clinical notes, and service requests.

Limited visibility

Managers may need a consolidated view of workload and activity.

SmartCare response: a shared application with role-specific screens and connected data.

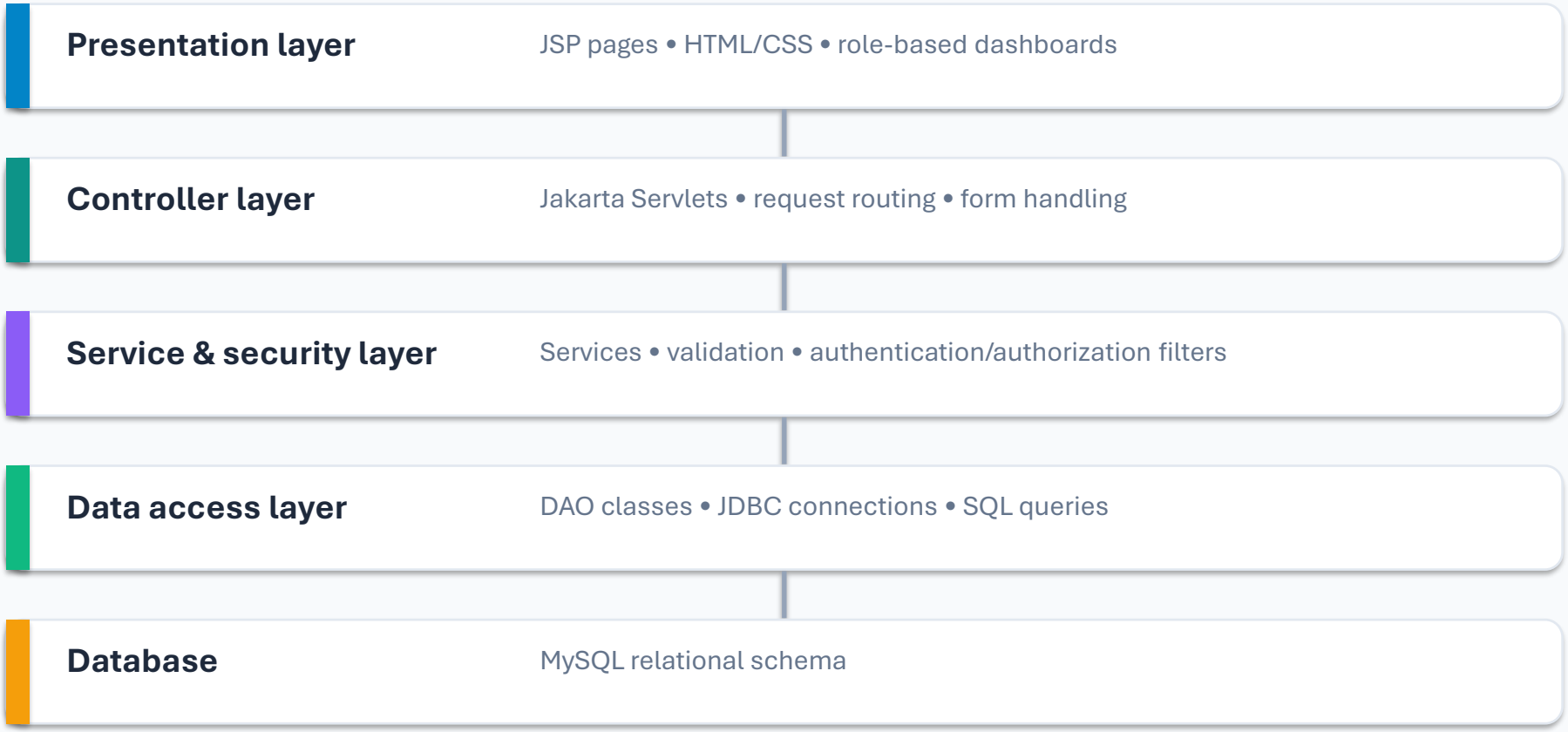
Objectives

The system focuses on practical workflow coverage rather than isolated screens.

01	**Centralize records**	Patients, doctors, departments, rooms, and admissions.
02	**Coordinate visits**	Appointment booking, status tracking, and consultation workflows.
03	**Support clinical tasks**	Medical records, prescriptions, vitals, and laboratory reports.
04	**Manage operations**	Pharmacy inventory, billing, payments, and reports.
05	**Protect access**	Authentication, authorization filters, and password hashing.

System architecture

A layered Java web architecture separates interface, request handling, business logic, and data.



Request → validation / authorization → business operation → database → response

Functional module map

The modules are connected through shared patient, user, appointment, and billing records.

Administration

Users • roles • departments • audit logs

Patient services

Registration • profile • appointments • feedback

Clinical care

Consultations • medical records • prescriptions

Nursing & lab

Vitals • laboratory tests • reports

Hospital operations

Admissions • rooms • pharmacy inventory

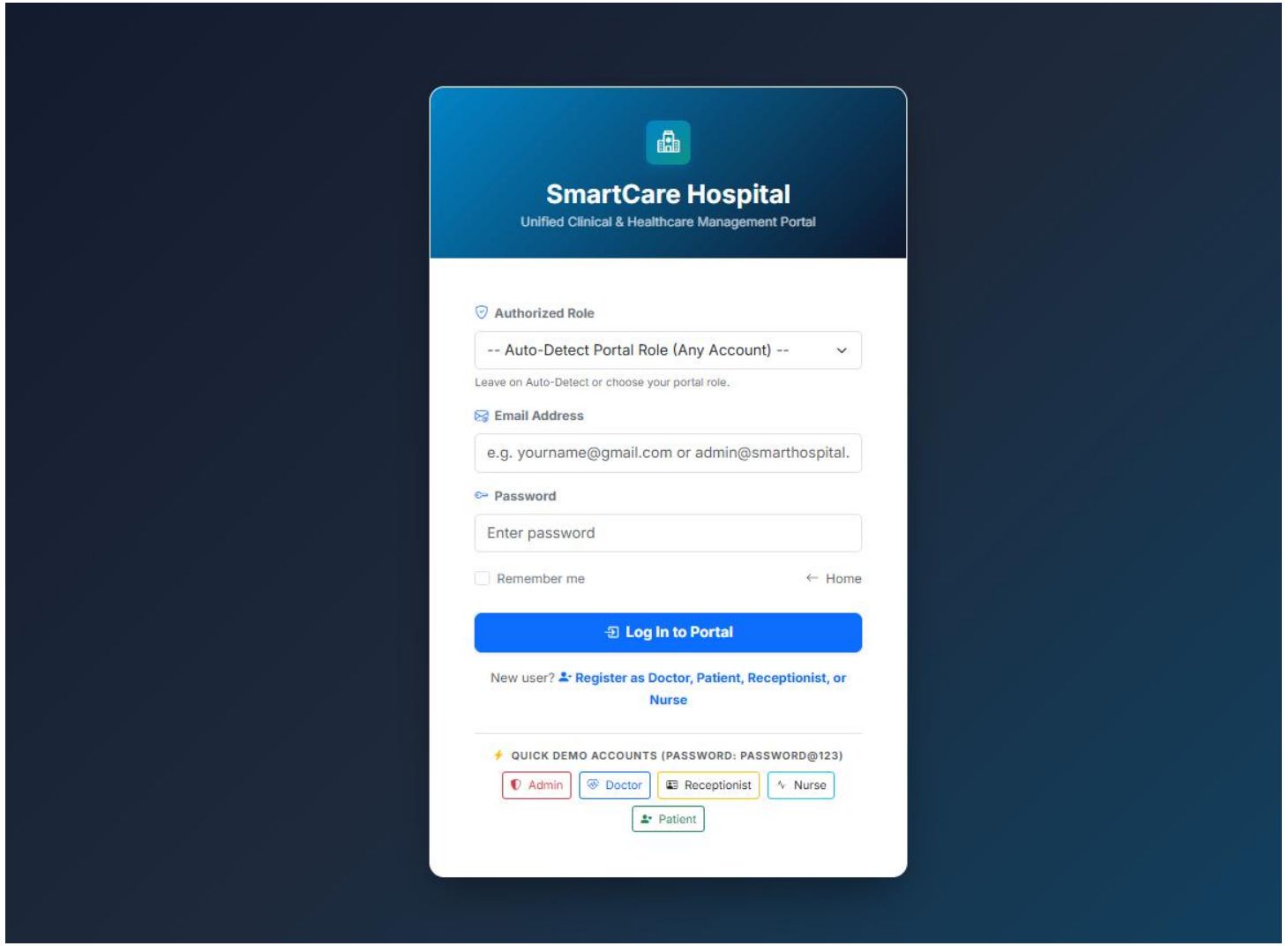
Finance & reports

Bills • payments • invoices • summaries

Access to each function is intended to depend on the signed-in role and authorization rules.

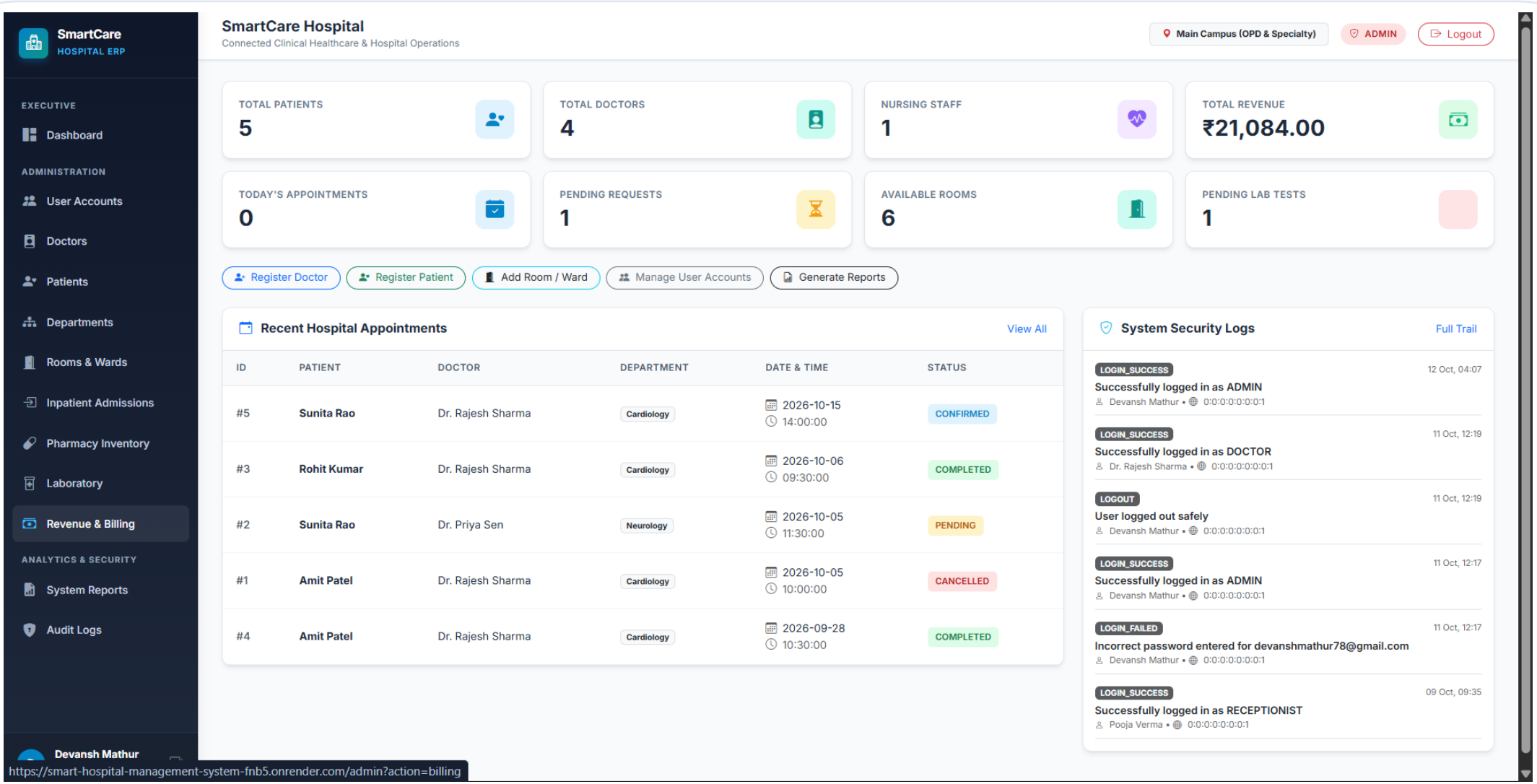
Application Screen: SmartCare Login

Actual login page from the SmartCare application.



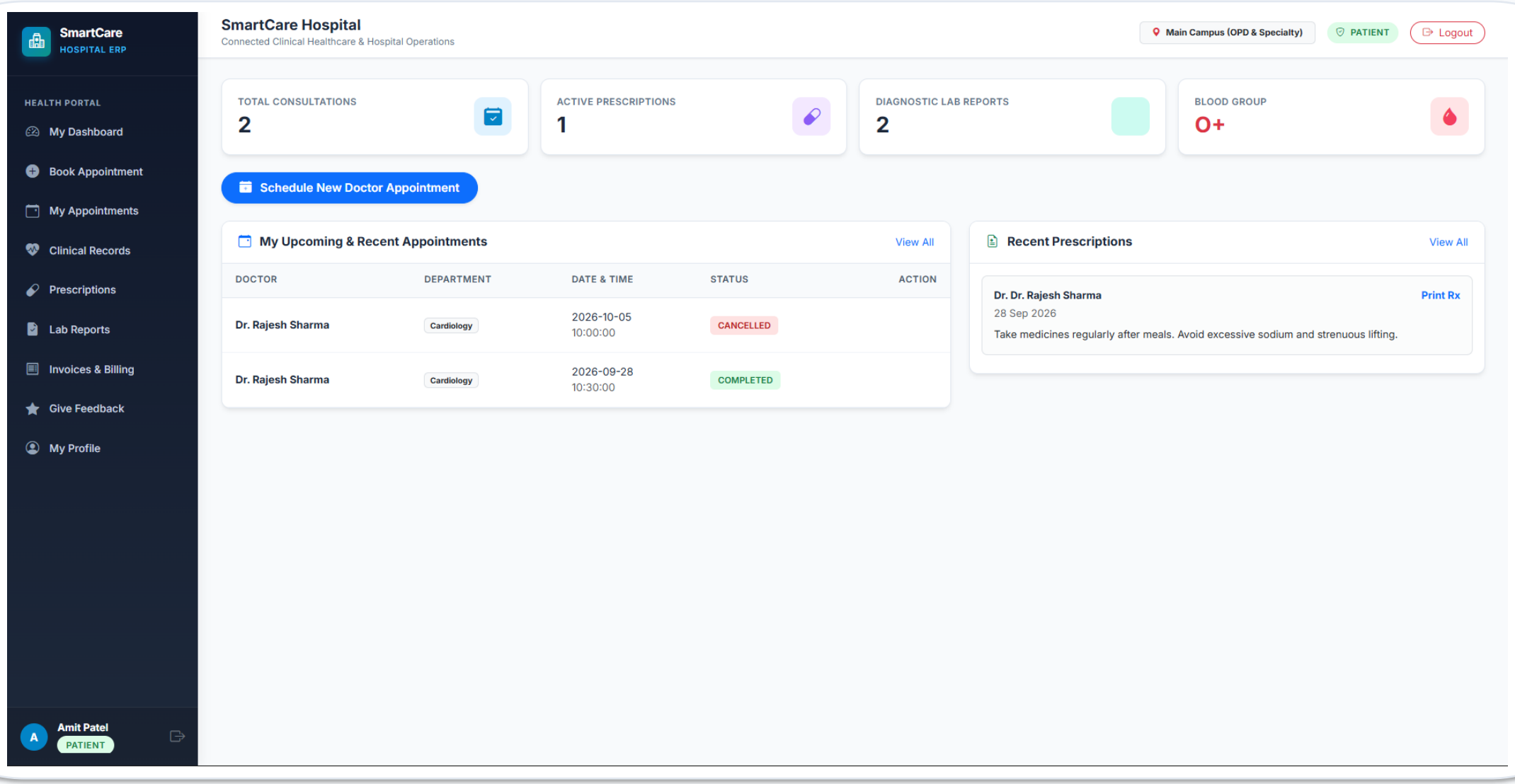
Application Screen: Administrator Dashboard

Administrator overview with hospital KPIs, appointment activity, and security logs.



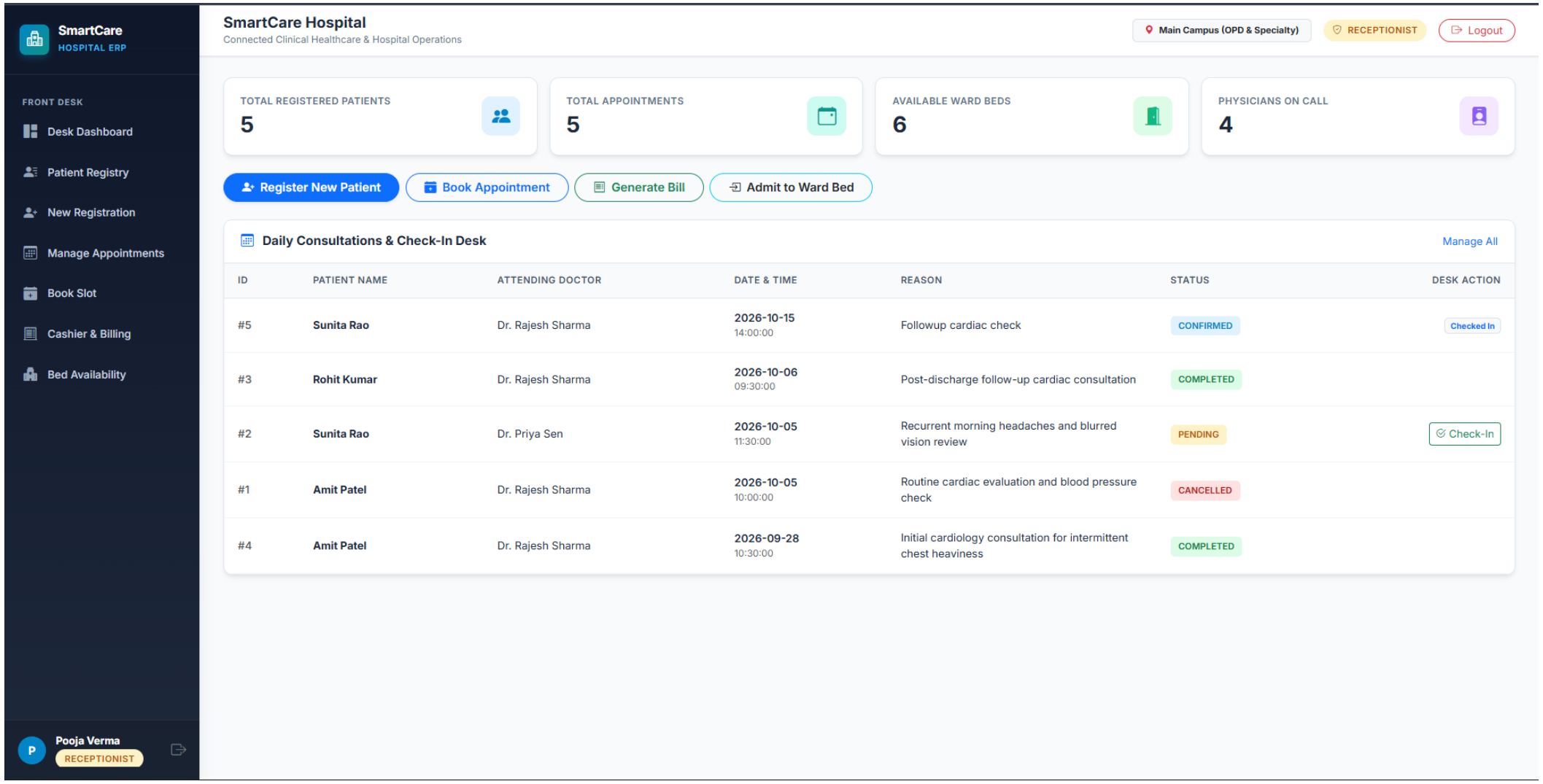
Application Screen: Patient Dashboard

Patient view with consultations, prescriptions, lab reports, and appointments.



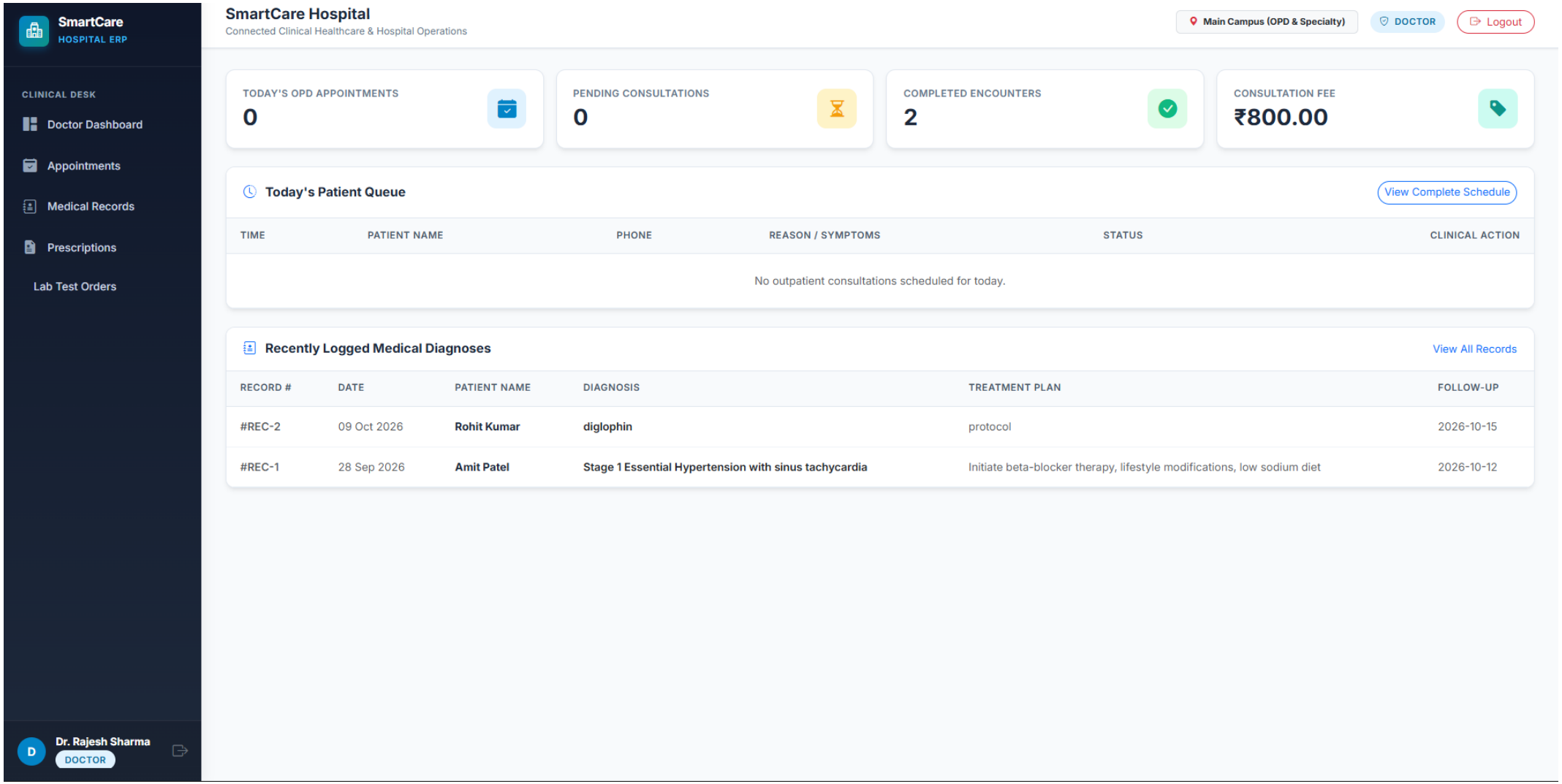
Application Screen: Reception Portal

Front-desk workflow for registration, appointment management, check-in, billing, and bed availability.



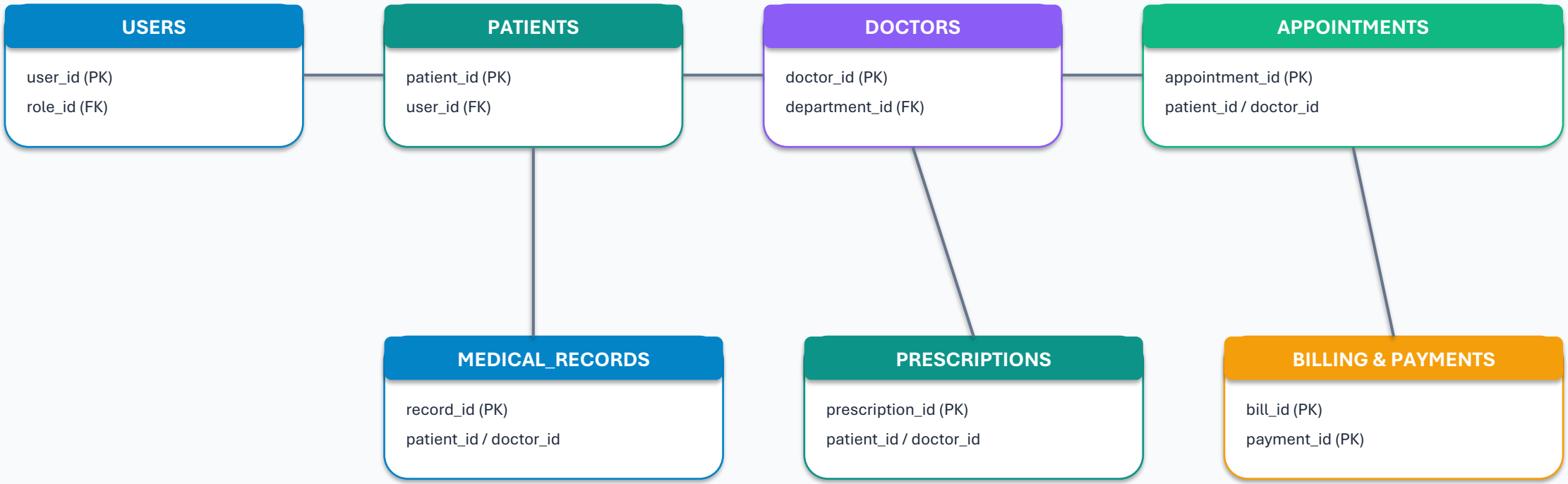
Application Screen: Doctor Portal

Doctor workflow for appointments, diagnoses, medical records, and clinical actions.



Database design: core entities

The relational schema links users and operational records through identifiers and foreign-key relationships.



Additional entities include departments, nurses, rooms, admissions, medicine inventory, lab tests, vitals, feedback, notifications, and audit logs.

Security and end-to-end workflow

A typical user request moves through authentication, authorization, validation, and persistence.

1

Sign in

Credentials submitted

2

Authorize

Role / session checked

3

Process

Servlet + service + validation

4

Persist

DAO / JDBC / MySQL

5

Respond

Updated view or status

Authentication

BCrypt password hashing is included.

Authorization

Filters protect restricted routes.

Validation

Input checks support safer data handling.

Implementation stack

Technology choices reflected in the provided project source archive.

Java 17

Application language and runtime

Jakarta Servlet / JSP

HTTP request handling and server-rendered pages

JDBC + DAO

Database connectivity and data-access separation

MySQL

Relational persistence for hospital entities

Maven WAR

Build and web application packaging

BCrypt + filters

Password hashing and access-control support

Source code also contains controller, service, DAO, model, utility, and JSP view layers.

Testing and validation approach

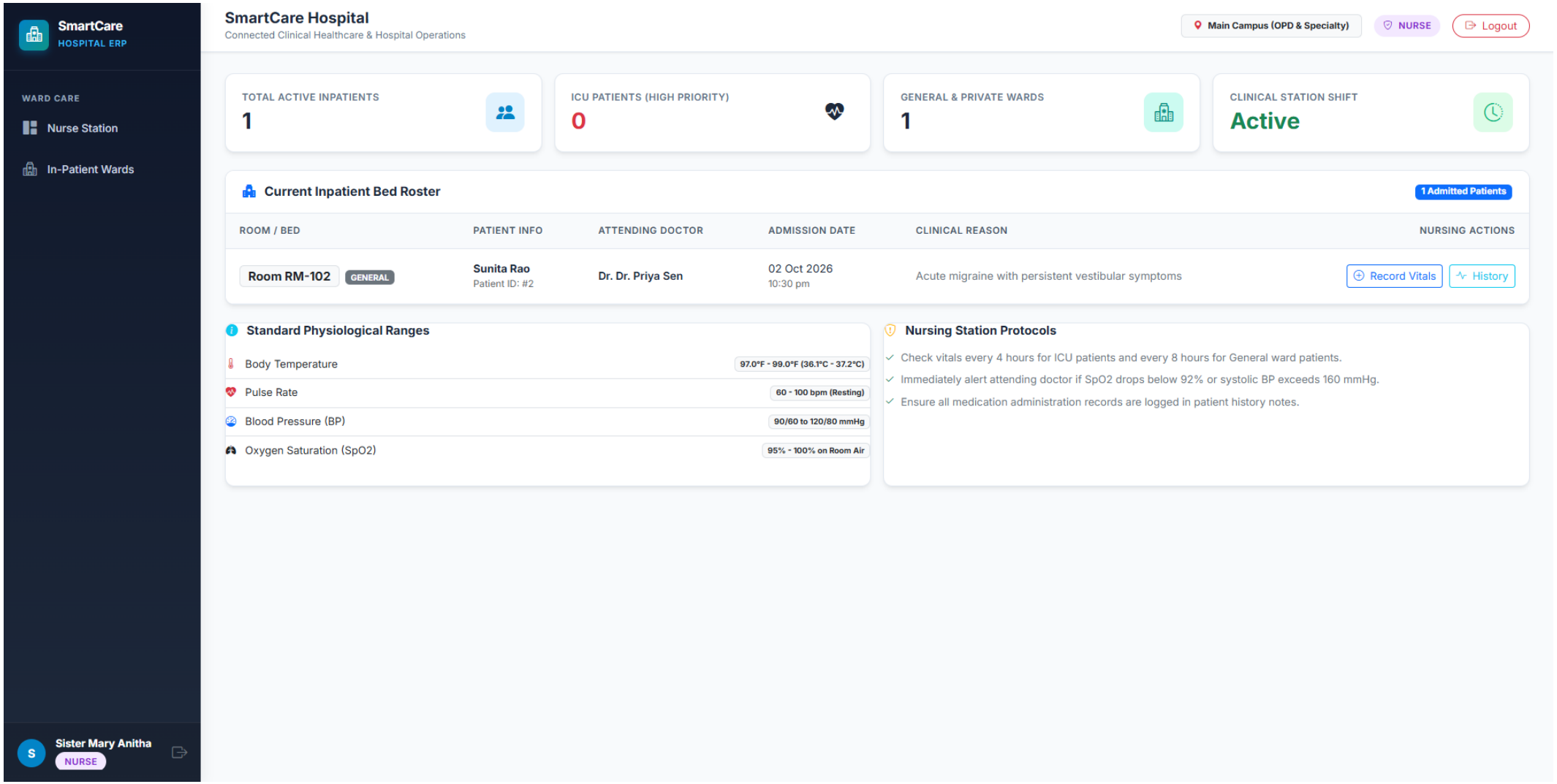
Suggested demonstration checklist based on the system modules and security controls.

Authentication	Valid / invalid credentials; logout; session expiry
Authorization	Restricted routes by role; unauthenticated access
Patient & doctor	Create, view, update, and search records
Appointments	Book, list, update status, and prevent invalid inputs
Clinical modules	Medical records, prescriptions, vitals, lab reports
Finance & operations	Billing, payments, admissions, inventory

Note: this is a proposed test checklist, not a claim that every test was executed.

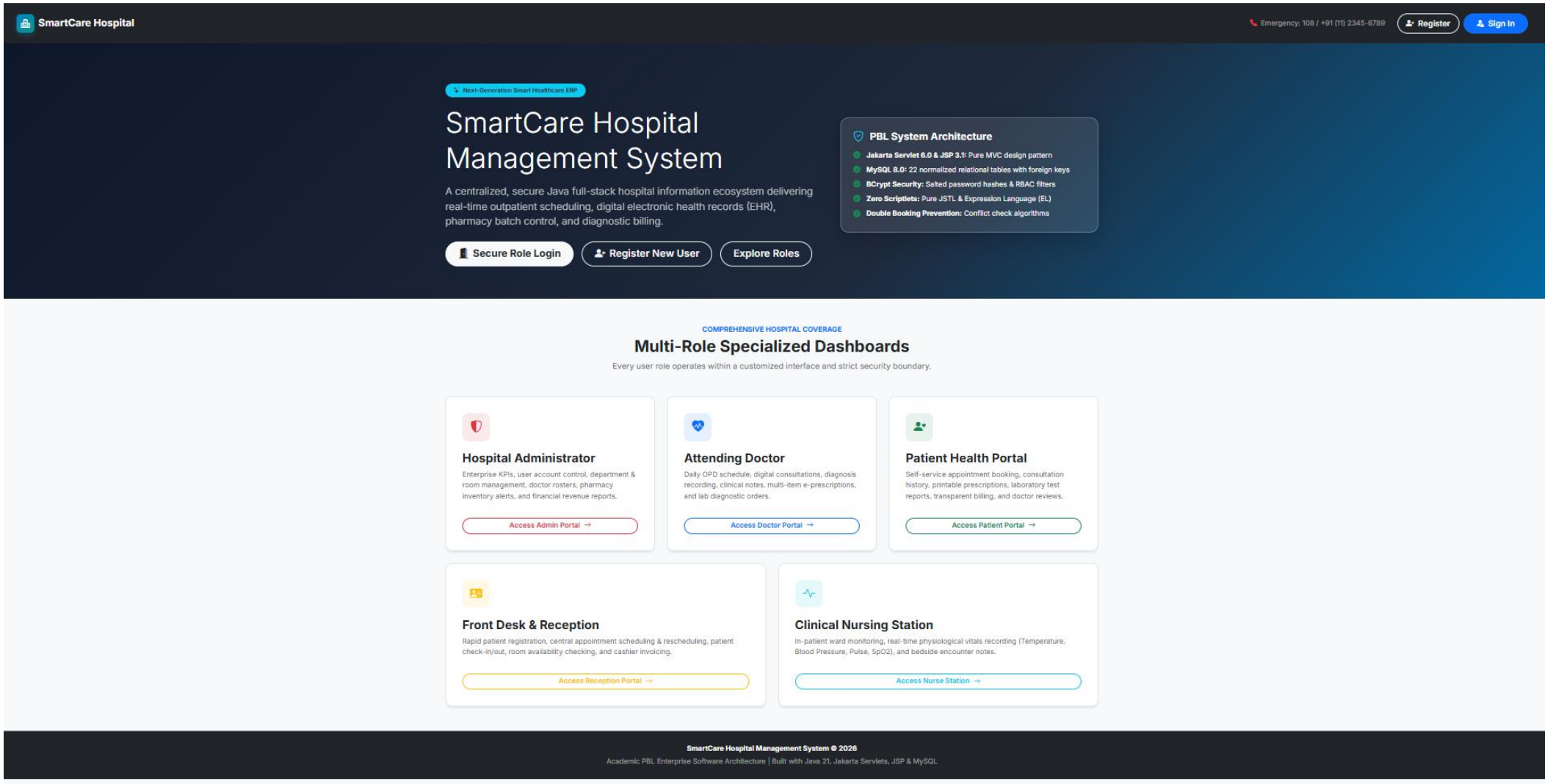
Application Screen: Nursing Station

Inpatient bed roster, patient vitals, ward details, and nursing protocols.



Application Screen: Public Home Page

Landing page with role-specific entry points for the hospital's main portals.



Conclusion

SmartCare connects essential hospital workflows through one role-based Java web application.

- Shared records across administration and clinical modules
- Layered architecture with JDBC and a MySQL database
- Authentication and role-based access controls
- Scope for future expansion: analytics, reminders, and integrations

THANK YOU

Questions & Discussion